

Satellite Collision AI — API Guide

Oct 4, 2026 · @Bob

Overview

The Debris Cloud Modeling API (v1.1) takes a satellite breakup event and its orbit. It returns the predicted cloud of untrackable debris (smaller than 10 cm) as a time series of 3D density voxels.

A neural network trained on historical breakups predicts corrections to the NASA Standard Breakup Model. Each correction comes as a p10/p50/p90 band. The physics engine then generates and propagates the fragments.

- **Base URL:** `http://localhost:8000` when you run it locally with uvicorn's defaults
- **Interactive docs:** `/docs` (Swagger UI, generated by FastAPI)
- **Format:** JSON in, JSON out. There is no authentication, and CORS allows only the origins listed in `CORS_ORIGINS`.

Running it locally

Install the dependencies, point the API at a trained model, and start uvicorn from `cloud_predictor/`.

1. Install the dependencies: `pip install -r cloud_predictor/ml/requirements.txt`.
The CPU build of torch is enough.
2. If you don't already have a model, train one:

```
python -m ml.train_breakup_scaler --events data/historical_breakups.csv \
    --specs data/gunter_satellites.json --fragments data/fragments.csv --
out models/estimator-v1/
```

3. Start the server:

```
cd cloud_predictor
BREAKUP_MODEL_DIR=models/estimator-v1 uvicorn app:app --port 8000
```

4. Check `GET /v1/health`. It should report `"model_loaded": true` after the first prediction call, because the model loads lazily.

Without `BREAKUP_MODEL_DIR`, the API still runs the uncorrected Standard Breakup Model (`corrections: "auto"` falls back to `neutral-sbm`). In that case `/v1/breakup-parameters` returns 503.

Variable	Default	Purpose
<code>BREAKUP_MODEL_DIR</code>	not set	Folder of the trained neural-net estimator
<code>SIMULATION_WORKER</code>	<code>1</code>	Set to <code>0</code> to skip the background worker that runs <code>/v1/simulations</code>
<code>SPACETIME_HOST</code>	<code>http://127.0.0.1:3000</code>	SpacetimeDB server used for stored simulations
<code>SPACETIME_DB</code>	<code>debris-tracker</code>	SpacetimeDB database name
<code>XAI_TOKEN</code>	not set	xAI key for <code>/v1/satellite-image</code> . It can also go in the repo-root <code>.env</code> .
<code>CORS_ORIGINS</code>	<code>http://localhost:5173,http://127.0.0.1:5173</code>	Comma-separated browser origins that are allowed to call the API

Endpoints

The main call is `POST /v1/debris-cloud`. It returns the full cloud in one response. For longer Monte Carlo runs, queue the job with `POST /v1/simulations` and poll it.

Method	Path	What it does
GET	<code>/v1/health</code>	Service status and whether a model is loaded
POST	<code>/v1/breakup-parameters</code>	The neural net's correction bands (p10/p50/p90) for one event. The body is the event object itself.
POST	<code>/v1/debris-cloud</code>	Runs the full model during the request: event + orbit in, density frames out

Method	Path	What it does
POST	<code>/v1/debris-cloud:publish</code>	Same as above, then streams the frames into SpacetimeDB. Also requires <code>event_id</code> and <code>sat_name</code> .
POST	<code>/v1/satellite-image</code>	An artist's impression of the target satellite from Grok Imagine (needs <code>XAI_TOKEN</code>). This is not model output.
POST	<code>/v1/simulations</code>	Queues a simulation (10 runs by default) and returns 202 with <code>id</code>
GET	<code>/v1/simulations</code>	Lists recent simulations and their status
GET	<code>/v1/simulations/{id}</code>	Status, progress, event metadata and frame times
GET	<code>/v1/simulations/{id}/frame?t_s=</code>	The cloud at a given time: the latest frame at or before <code>t_s</code>
GET	<code>/v1/simulations/{id}/frames?start_s=&end_s=</code>	Every frame in a time range
GET	<code>/v1/simulations/{id}/particles</code>	Representative fragment orbits, for smooth animation
DELETE	<code>/v1/simulations/{id}</code>	Deletes a simulation (returns 204)

Request body

`/v1/debris-cloud` and `/v1/simulations` take the same body. Only `event` and `orbit` are required.

Field	Type	Default	Allowed values
<code>event</code>	object	required	See the event table below

Field	Type	Default	Allowed values
<code>orbit</code>	object	required	<code>{alt_km, inc_deg, raan_deg?, arg_lat_deg?}</code> for a circular orbit, or <code>{r_km: [x,y,z], v_km_s: [vx,vy,vz]}</code> for a state vector. <code>alt_km</code> is 150–50,000 and <code>inc_deg</code> is 0–180.
<code>corrections</code>	string or object	"auto"	"auto" samples the neural-net bands, "p50" uses the median in every run, and "neutral" uses the plain SBM. An object such as <code>{"n_multiplier": 1.4}</code> sets values directly.
<code>times_s</code>	number[]	0 s, 10 min, 30 min, then hourly to 48 h	1–100 snapshot times, each from 0 to 2,592,000 s (30 days)
<code>n_runs</code>	int	1 (sync), 10 (simulations)	1–30 Monte Carlo runs
<code>k</code>	int	10,000	100–100,000 sampled fragments
<code>voxel_km</code>	number	20	1–500 km voxel edge
<code>sigma_voxels</code>	number	1	0–5 smoothing in voxels
<code>max_voxels</code>	int	1000	10–2,000 voxels per frame
<code>seed</code>	int	0	0 to $2^{31}-1$

Event object

Field	Type	Notes
<code>event_type</code>	string	"explosion" or "collision"
<code>epoch</code>	string	ISO 8601. A time with no zone is read as UTC.
<code>target.object_class</code>	string	payload, rocket_body or debris
<code>target.dry_mass_kg</code>	number	Must be > 0

Field	Type	Notes
<code>target.propellant_mass_kg</code>	number	Optional, default 0
<code>target.structure_material</code>	string	Optional: <code>aluminum</code> , <code>cfrp</code> , <code>mixed</code> or <code>unknown</code>
<code>target.solar_array_area_m2</code> , <code>bus_volume_m3</code> , <code>mli_fraction</code> , <code>launch_year</code> , <code>bus_family</code> , <code>norad_id</code>	various	Optional. <code>mli_fraction</code> is 0-1. Fields you leave out are imputed.
<code>impactor</code>	object	Required for collisions and must be null for explosions: <code>{mass_kg, v_rel_km_s, structure_material?}</code> . <code>v_rel_km_s</code> must be above 0 and at most 20.
<code>explosion_cause</code>	string	Optional: <code>propulsion</code> , <code>battery</code> , <code>deliberate</code> or <code>unknown</code>

Response from `/v1/debris-cloud`

Field	Meaning
<code>model_version</code>	The estimator version, <code>neutral-sbm</code> or <code>overrides</code>
<code>corrections</code>	The single-run correction values that were applied
<code>parameters</code>	The neural net's full band prediction (null when no model was used)
<code>derived</code>	<code>emr_j_per_g</code> , <code>is_catastrophic</code> , <code>sbm_mass_param</code>
<code>n_runs</code> , <code>fragments_total</code> , <code>voxel_km</code>	Run count, fragments between 2 mm and 10 cm, and voxel size
<code>frames[]</code>	One per time: <code>t_s</code> , <code>in_orbit_fraction</code> , <code>n_voxels</code> , <code>peak_density</code> , <code>voxels</code>

Each voxel is `[x, y, z, density]` in ECI km and fragments/km³, densest first. When `n_runs > 1`, each voxel is `[x, y, z, mean, p10, p50, p90]`. Frames from stored simulations always use the 7-column form.

Examples

Each example models a 1,000 kg carbon-fiber satellite at 780 km, struck by a 50 kg object at 10 km/s. The runnable version is `cloud_predictor/examples/api_example.py`.

One synchronous call (curl)

```
curl -X POST http://localhost:8000/v1/debris-cloud \
  -H "Content-Type: application/json" \
  -d '{
    "event": {
      "event_type": "collision",
      "epoch": "2026-10-04T12:00:00Z",
      "target": {"object_class": "payload", "dry_mass_kg": 950,
"propellant_mass_kg": 50,
      "structure_material": "cfrp", "mli_fraction": 0.5},
      "impactor": {"mass_kg": 50, "v_rel_km_s": 10}
    },
    "orbit": {"alt_km": 780, "inc_deg": 86.4},
    "times_s": [0, 21600, 172800]
  }'
```

Python

```
import httpx

API = "http://127.0.0.1:8000"
request = {
    "event": {
        "event_type": "collision",
        "epoch": "2026-10-04T12:00:00Z",
        "target": {"object_class": "payload", "dry_mass_kg": 950,
"propellant_mass_kg": 50,
        "structure_material": "cfrp", "mli_fraction": 0.5},
        "impactor": {"mass_kg": 50, "v_rel_km_s": 10},
    },
    "orbit": {"alt_km": 780, "inc_deg": 86.4},
```

```

    "times_s": [0, 6 * 3600, 48 * 3600],
}

result = httpx.post(f"{API}/v1/debris-cloud", json=request,
timeout=120).json()
print(result["model_version"], result["fragments_total"])
for frame in result["frames"]:
    x, y, z, density = frame["voxels"][0][:4] # densest voxel
    print(frame["t_s"], frame["in_orbit_fraction"], density)

```

Queued simulation

1. Send `POST /v1/simulations` with the same body, plus `"n_runs": 10` if you want something other than the default. The response is `202 {"id": 7, "status": "...", "links": {...}}`.
2. Poll `GET /v1/simulations/7` until `status` is `done`. `progress` shows how far along it is.
3. Read the cloud with `GET /v1/simulations/7/frame?t_s=21600` (a single time) or `GET /v1/simulations/7/frames` (every frame).

Just the neural-net corrections

Send the event object on its own (no `orbit` wrapper) to `POST /v1/breakup-parameters`. The response holds p10/p50/p90 bands for `n_multiplier`, `slope_delta`, `am_mu_shift`, `am_sigma_scale`, `dv_mu_shift` and `dv_sigma_scale`. It also includes `imputed_fields`, `fallback` and `warnings`.

Errors and limits

A validation error (422) lists every bad field at once, for example `{"detail": [{"loc": "orbit.alt_km", "msg": "must be a number in [150, 50000]"}]}`. Every other error returns `{"detail": "<message>"}`.

Status	When
202	A simulation was queued
204	A simulation was deleted
404	Unknown simulation id

Status	When
409	The simulation exists but isn't <code>done</code> yet, or it failed
422	Invalid JSON, a missing field, or a value out of range
502	SpacetimeDB or the xAI image service failed
503	No model is loaded (<code>BREAKUP_MODEL_DIR</code> is unset), or <code>XAI_TOKEN</code> is missing

- The request limits keep each call bounded: up to 100,000 fragments, 30 runs, 100 snapshot times, 2,000 voxels per frame, and a 30-day horizon.
- A synchronous `/v1/debris-cloud` call with many runs can take tens of seconds. Use a client timeout of about 120 s, or use `/v1/simulations` instead.
- Out-of-range values in a `corrections` object are clamped to the model's bounds instead of rejected. For example, `n_multiplier` is clamped to 0.003–5.
- The `/v1/simulations` endpoints need a running SpacetimeDB and the background worker.